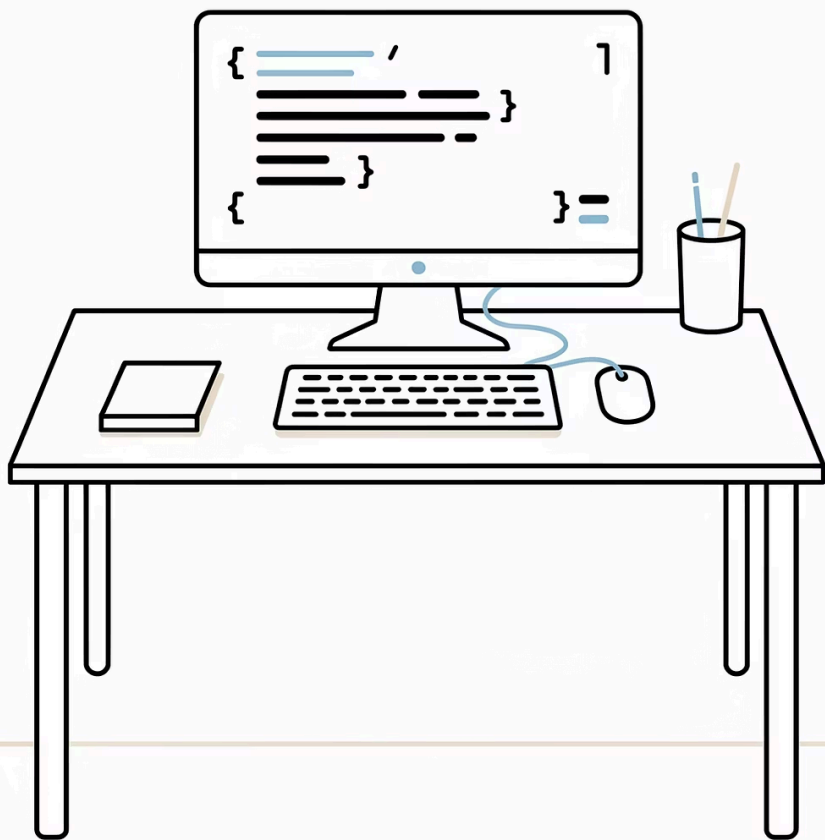




Criando um Sistema Web Simples

Login + Página Inicial com HTML, CSS e JavaScript

AULA DE DESIGN PARA WEB



Classes CSS — Como Usar e Para Que Servem

Conhecer as classes e seletores CSS do projeto é essencial para personalizar e expandir a interface. Veja abaixo as principais classes, sua finalidade e como aplicá-las corretamente.

body

Seletor global que define a fonte padrão (Arial), zera margens e aplica a cor de fundo (#f5f5f5) em toda a página.

#login-container

Centraliza o formulário de login na tela, define largura fixa de 300px, fundo branco, padding interno e cantos arredondados com border-radius: 8px.

#login-container input

Estiliza os campos de texto do formulário — largura de 90%, padding confortável e espaçamento entre os campos.

#login-container button

Botão de login com fundo verde (#4CAF50), texto branco, largura total e cursor pointer para indicar que é clicável.

.menu

Container do header com fundo escuro (#333), texto branco e display: flex para distribuir o título e os links nas extremidades opostas.

.menu a

Links do menu com cor clara (#fff), sem sublinhado e com espaçamento à esquerda para separação visual entre os itens.

.conteudo

Área principal do sistema após o login. Aplica padding de 20px para afastar o conteúdo das bordas da tela.

.rodape

Rodapé fixo na parte inferior da tela (position: fixed; bottom: 0), com fundo escuro, texto branco e flex para alinhar texto e links lado a lado.



Dica: Seletores com # (como #login-container) identificam elementos únicos na página. Seletores com . (como .menu) podem ser aplicados a múltiplos elementos. Use IDs para componentes únicos e classes para estilos reutilizáveis.



Estilizando Botões com CSS

Um botão bem estilizado melhora a experiência do usuário e transmite profissionalismo. Com CSS, você pode transformar um botão simples em um elemento visualmente atraente e interativo.

Do Básico ao Estilizado

Veja a evolução de um botão simples com cada propriedade CSS adicionada:

```
button {
  padding: 12px 24px;
  background-color: #4CAF50;
  color: white;
  border: none;
  border-radius: 8px;
  font-size: 16px;
  font-weight: bold;
  cursor: pointer;
  letter-spacing: 1px;
  transition: all 0.3s ease;
  box-shadow: 0 4px 6px
    rgba(0, 0, 0, 0.2);
}

button:hover {
  background-color: #45a049;
  transform: translateY(-2px);
  box-shadow: 0 6px 12px
    rgba(0, 0, 0, 0.3);
}

button:active {
  transform: translateY(0);
  box-shadow: none;
}
```

O que cada propriedade faz

padding: 12px 24px

Define o espaço interno do botão — 12px na vertical e 24px na horizontal, deixando-o mais largo e confortável.

border-radius: 8px

Arredonda os cantos do botão. Use 50px para um botão totalmente arredondado (pill button).

transition: all 0.3s ease

Cria uma animação suave em todas as propriedades ao passar o mouse, tornando a interação mais elegante.

box-shadow

Adiciona sombra ao botão, criando profundidade e sensação de elevação na interface.

:hover e :active

Pseudo-classes que aplicam estilos quando o usuário passa o mouse ou clica no botão, dando feedback visual.

transform: translateY(-2px)

Move o botão 2px para cima ao passar o mouse, simulando um efeito de "levantar" ao ser tocado.



Dica: Combine transition com :hover para criar botões com animações suaves. Pequenos detalhes como sombra e movimento fazem grande diferença na percepção de qualidade da interface.

O que você vai aprender hoje

Ao final desta aula, você será capaz de construir um sistema web completo do zero — com tela de login funcional, validação via JavaScript e uma página inicial com menu superior.

Tela de Login

Criar um formulário de login visualmente organizado com campos de usuário e senha.

Validação JS

Usar JavaScript para verificar credenciais e controlar a exibição das telas.

Menu Superior

Criar um header reutilizável com links de navegação e carregá-lo dinamicamente.

Separação de Arquivos

Organizar HTML, CSS e JS em arquivos separados, seguindo o padrão de mercado.



Estrutura do Projeto

Antes de escrever qualquer linha de código, é fundamental organizar os arquivos do projeto. Cada arquivo tem uma responsabilidade única e bem definida — essa separação é a base de qualquer desenvolvimento web profissional.

Arquivos do Projeto

01

index.html

Tela principal contendo o formulário de login e a área do sistema pós-autenticação.

02

header.html

Menu de navegação superior, carregado dinamicamente após o login.

03

javascript.js

Toda a lógica da aplicação: validação, manipulação do DOM e carregamento dinâmico.

04

estilo.css

Toda a aparência visual: cores, espaçamentos, tipografia e layout responsivo.

Por que essa organização?

Separar os arquivos traz benefícios reais no dia a dia do desenvolvedor:

→ **Organização**

Cada arquivo tem um propósito claro, facilitando a leitura e manutenção do código.

→ **Reutilização**

O header pode ser reaproveitado em múltiplas páginas sem duplicar código.

→ **Padrão de Mercado**

Empresas e equipes profissionais seguem essa estrutura em projetos reais.



Conceitos Fundamentais

Esta aula não é apenas sobre escrever código — é sobre entender os pilares do desenvolvimento front-end moderno. Esses conceitos são usados em todos os projetos web profissionais.



Separação de Responsabilidades

HTML estrutura o conteúdo, CSS cuida da aparência e JS gerencia o comportamento. Misturar os três em um único arquivo cria código difícil de manter.



Eventos (onclick)

Eventos permitem que o código reaja a ações do usuário — como um clique em um botão — executando funções específicas no momento certo.



Manipulação do DOM

O DOM (Document Object Model) é a representação da página em memória. Com JavaScript, você acessa e modifica elementos em tempo real, sem recarregar a página.



Estrutura de Layout Web

Aprender a construir um layout com header, área de conteúdo e containers é a base para qualquer interface moderna.

index.html — A Estrutura Principal

O `index.html` é o coração do projeto. Ele contém dois blocos principais: o formulário de login (visível no início) e a área do sistema (oculta até que o login seja validado com sucesso).

Bloco de Login

```
<div id="login-container">
  <h2>Login</h2>
  <input type="text" id="usuario"
    placeholder="Usuário">
  <input type="password"
    id="senha"
    placeholder="Senha">
  <button onclick="fazerLogin()">
    Entrar
  </button>
  <p id="erro"></p>
</div>
```

Bloco do Sistema

```
<div id="sistema"
  style="display: none;">
  <div id="header"></div>
  <div class="conteudo">
    <h1>Bem-vindo!</h1>
    <p>Login realizado
      com sucesso.</p>
  </div>
</div>
```

 **Dica:** O atributo `style="display: none;"` no bloco do sistema é o que o torna invisível inicialmente. O JavaScript vai removê-lo após um login bem-sucedido.

○○○

```
</ | LOGIN: _____ }
| C9DBFS: ____ } }
| _____ { _____ - }
| _____ }
| = { _____ }
| THNG:: _____ { _____ }
| _____ } } }
| _____ } }
| { _____ }
| { _____ { LOU _____ } }
|
| = 7A88CEA: _____ }
| { _____ }
| E8DCC7 { _____ }
| { _____ { _____ } }
| { _____ }
| { _____ }
|
>> |
```

header.html — O Menu de Navegação

O `header.html` é um fragmento HTML independente que representa o menu superior da aplicação. Ele não é uma página completa — não tem `<html>` nem `<body>` — é apenas um componente reutilizável que será injetado no `index.html` via JavaScript.

Código do Header

```
<div class="menu">
  <span>Meu Sistema</span>
  <div class="links">
    <a href="#">Início</a>
    <a href="#">Perfil</a>
    <a href="#">Sair</a>
  </div>
</div>
```

Por que um arquivo separado?

Separar o menu em um arquivo próprio traz vantagens práticas importantes:

- Se você tiver 10 páginas, atualiza o menu em **um único lugar**
- O componente pode ser **reaproveitado** em qualquer parte do projeto
- Simula o conceito de **componentes** usado em frameworks como React e Vue
- Mantém o `index.html` mais **limpo e focado**

javascript.js — A Lógica da Aplicação

O arquivo JavaScript é onde a magia acontece. Ele contém duas funções que trabalham juntas: uma para validar o login e outra para carregar o menu dinamicamente.

Função fazerLogin()

```
function fazerLogin() {
  const usuario =
    document.getElementById(
      "usuario").value;
  const senha =
    document.getElementById(
      "senha").value;
  const erro =
    document.getElementById("erro");

  if (usuario === "admin"
    && senha === "123") {
    document.getElementById(
      "login-container"
    ).style.display = "none";
    document.getElementById(
      "sistema"
    ).style.display = "block";
    carregarHeader();
  } else {
    erro.innerText =
      "Usuário ou senha inválidos!";
  }
}
```

Função carregarHeader()

```
function carregarHeader() {
  fetch("header.html")
    .then(response =>
      response.text())
    .then(data => {
      document.getElementById(
        "header"
      ).innerHTML = data;
    });
}
```

 **O que é fetch()?** É a forma moderna do JavaScript para carregar arquivos ou consumir APIs. Aqui, ele busca o conteúdo do `header.html` e injeta diretamente na div `#header`.

estilo.css — O Visual da Aplicação

O CSS transforma a estrutura HTML em uma interface agradável. Cada seletor tem um papel específico: estilizar o login, o menu ou o conteúdo principal.

Estilo do Login

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  background-color: #f5f5f5;
}

#login-container {
  width: 300px;
  margin: 100px auto;
  padding: 20px;
  background: white;
  border-radius: 8px;
  text-align: center;
}

#login-container input {
  width: 90%;
  padding: 10px;
  margin: 10px 0;
}

#login-container button {
  padding: 10px;
  width: 100%;
  background: #4CAF50;
  color: white;
  border: none;
  cursor: pointer;
}
```

Estilo do Menu e Conteúdo

```
.menu {
  background: #333;
  color: white;
  padding: 15px;
  display: flex;
  justify-content:
    space-between;
}

.menu a {
  color: white;
  margin-left: 15px;
  text-decoration: none;
}

.conteudo {
  padding: 20px;
}
```

border-radius: 8px

Cria cantos arredondados no container de login.

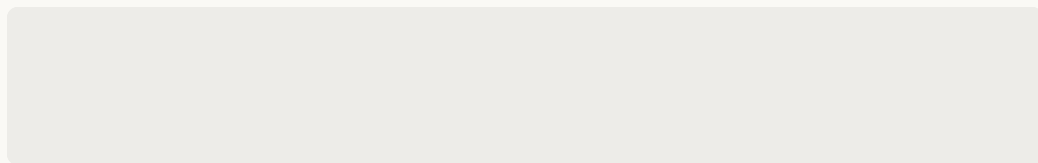
display: flex

Distribui os itens do menu horizontalmente.

footer.html — O Rodapé da Aplicação

O rodapé é um componente reutilizável, assim como o header. Ele será carregado dinamicamente via JavaScript e exibido na parte inferior da página após o login.

Código do Footer



© 2024 Meu Sistema. Todos os direitos reservados.

[Política de Privacidade](#) [Suporte](#) [Contato](#)

Estilo CSS do Footer

```
.rodape {  
  background: #333;  
  color: white;  
  padding: 15px;  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  position: fixed;  
  bottom: 0;  
  width: 100%;  
}  
  
.rodape a {  
  color: #ccc;  
  margin-left: 15px;  
  text-decoration: none;  
}
```

 Dica: Assim como o `header.html`, o `footer.html` é carregado dinamicamente com `fetch()`. Adicione a função `carregarFooter()` no `javascript.js` e chame-a junto com `carregarHeader()` após o login bem-sucedido.